Software Testing

All Types of Testing Explained with Real Examples

Detailed student notes: levels, methods, functional and non-functional testing, test design techniques, smoke vs sanity vs regression, bug life cycle, and interview questions.

Part	Topic
1	Basics: what is testing, key terms, STLC, testing pyramid, running example
2	Levels of testing: unit, integration, system, acceptance
3	Testing approaches: manual, automation, black box, white box, grey box
4	Functional testing types: smoke, sanity, regression, retesting, exploratory, ad-hoc, API, UI and more
5	Non-functional testing types: performance, security, usability, compatibility, reliability and more
6	Test design techniques: equivalence partitioning, BVA, decision table, state transition, coverage
7	Other important testing types: alpha, beta, A/B, mutation, fuzz, chaos, static, mobile, and more
8	Smoke vs Sanity vs Regression vs Retesting (comparison)
9	Bug life cycle, severity vs priority, test case format
10	Quick revision table and interview questions

Part 1: Basics of Software Testing

Software testing is the process of checking a software application to find defects (bugs) and to confirm that it does what the customer expects. A tester asks two questions: *Are we building the product right?* (verification) and *Are we building the right product?* (validation).

Why testing is needed

- To find bugs before the customer finds them (a bug found in production costs many times more to fix).
- To make sure the product is safe, reliable, secure and fast enough.
- To build customer trust. Example: if a banking app deducts money twice, customers will leave.
- To meet legal and business rules (for example payment and data-privacy rules).

Key terms students must know

Term	Meaning	Simple example
Error (mistake)	A human mistake made by a developer or analyst.	Developer types **>** instead of **>=** in code.
Defect / Bug	The fault in the software caused by the error.	Discount is not applied for an order of exactly Rs. 1000.
Failure	The wrong behaviour users actually see when the bug runs.	Customer with a Rs. 1000 order sees no discount.
Test case	Steps + input + expected result to check one thing.	Login with valid user and password; expect home page.
Test scenario	A high-level thing to test (one scenario has many test cases).	Verify login functionality.
Test data	Input values used while testing.	Username, password, card number, PIN code.
Verification	Checking documents and code without running them (reviews).	Reading requirement document for missing rules.
Validation	Running the software to check it meets user needs.	Placing a real test order in the app.
QA vs QC	QA = process to prevent defects. QC = finding defects in the product.	Defining review rules (QA); executing test cases (QC).

Software Testing Life Cycle (STLC)

Testing is not one activity at the end. It follows phases: **1) Requirement analysis** (understand what to test) -> **2) Test planning** (scope, tools, schedule, people) -> **3) Test case design** (write scenarios, cases and data) -> **4) Test environment setup** (servers, devices, builds) -> **5) Test execution** (run tests, log defects) -> **6) Test closure** (reports, lessons learned).

The testing pyramid

A healthy project has **many** fast unit tests at the bottom, **some** integration/API tests in the middle, and **few** slow end-to-end UI tests at the top. Reason: small tests are cheap, quick and point exactly to the failing code; UI tests are slow and fragile.

Layer	How many	Speed	Cost to maintain
UI / End-to-end tests	Few	Slow	High
Integration / API tests	Medium	Medium	Medium
Unit tests	Many	Very fast	Low

The running example used in these notes

> **ShopEasy:** an online shopping website and mobile app. Features: **Register/Login**, **Search products**, **Add to cart**, **Apply coupon**, **Pay by card/UPI**, **Order tracking**, and **Email/SMS notification**. Most examples below use ShopEasy so you can see how each testing type applies to one product. A few examples use an ATM, a train-ticket booking site and a UPI payment app for variety.

Part 2: Levels of Testing

Levels describe **how big a piece** of the software we test: from one function up to the whole system with real users.

2.1 Unit Testing

What it is: Testing the smallest piece of code (a function, method or class) in isolation. Done by **developers**, usually automated (JUnit, TestNG, PyTest, NUnit, Jest).

When it is done: While or right after coding, before the code is merged.

> **Real example:** ShopEasy has a function *calculateDiscount(amount)*: 10% off for orders of Rs. 1000 or more. Unit tests: *calculateDiscount(999)* must return 0, *calculateDiscount(1000)* must return 100, *calculateDiscount(2000)* must return 200. If the developer wrote **>** instead of **>=**, the test for 1000 fails immediately.

- Fast, cheap, finds bugs earliest.
- Uses stubs/mocks to replace database or payment gateway.
- Developers also measure code coverage here.

2.2 Integration Testing

What it is: Testing that two or more modules work correctly **together**: data flows, interfaces and calls between modules.

When it is done: After unit testing, when modules are combined.

> **Real example:** Cart module and Payment module are each fine alone. Integration test: add 2 items (Rs. 1500 total) to the cart, click Pay; the Payment module must receive exactly Rs. 1500, not Rs. 0 or Rs. 15. Another: after payment success, the Order module must create the order and the Notification module must send the SMS.

- **Big Bang:** combine all modules at once, then test (simple, but hard to find which module failed).
- **Top-down:** test main modules first, use *stubs* for lower modules not ready yet.
- **Bottom-up:** test low-level modules first, use *drivers* to call them.
- **Sandwich (hybrid):** top-down and bottom-up together.

2.3 System Testing

What it is: Testing the **complete, integrated application** as one system against the full requirements, in an environment like production. Done by the independent QA team. It includes both functional and non-functional checks.

When it is done: After all modules are integrated and the build is stable.

> **Real example:** QA runs a full customer journey on ShopEasy: register -> login -> search 'running shoes' -> add to cart -> apply coupon -> pay by UPI -> receive confirmation email -> track order. Also checks speed, security and behaviour on Chrome, Firefox and Android.

2.4 Acceptance Testing (UAT)

What it is: Final testing by the **customer or business users** to decide whether they accept the product for release. It checks business needs, not code details.

When it is done: At the end, just before release.

> **Real example:** Before launch, ShopEasy's business team (store managers, customer-care staff) use the system with real business scenarios: process a return, issue a refund, create a festival-sale coupon. If they say 'this matches our needs', the product is accepted.

- **Alpha testing:** by internal staff / selected users at the company's own site.
- **Beta testing:** by real users at their own location before general release.
- **Contract / Regulatory acceptance:** checks the contract terms or legal rules.
- **Operational acceptance (OAT):** checks backup, recovery, maintenance and monitoring are ready.

Level	Who tests	What is tested	Typical tools
Unit	Developer	One function/class	JUnit, PyTest, Jest
Integration	Developer / tester	Module-to-module interfaces	Postman, REST Assured
System	QA team	Whole application	Selenium, Cypress, manual
Acceptance	Customer / business	Business readiness	Manual, UAT checklist

Part 3: Testing Approaches (How We Test)

3.1 Manual Testing

What it is: A human tester executes test cases step by step, without scripts, and compares actual with expected results.

> **Real example:** A tester opens ShopEasy on a phone, tries to place an order, and notices that the 'Pay' button overlaps the price text. A script would not notice this visual problem easily; a human sees it at once.

- Best for exploratory, usability and one-time tests.
- Slow and repetitive for large regression suites.

3.2 Automation Testing

What it is: Using tools and scripts to run tests automatically and compare results.

> **Real example:** A Selenium script logs in to ShopEasy, searches a product, adds it to the cart and checks the cart total - every night in 2 minutes. Doing the same 500 times manually would take days.

- Best for regression, performance, data-driven and repeated tests.
- Tools: Selenium, Cypress, Playwright, Appium (mobile), JMeter (performance), Postman (API).
- Not everything should be automated: new, unstable or one-time features are better tested manually first.

3.3 Black Box Testing

What it is: Testing only **inputs and outputs** without seeing the code. The tester does not need programming knowledge.

> **Real example:** Enter a coupon code 'SAVE10' on the cart page and check the price drops by 10%. The tester does not care how the coupon logic is written.

- Techniques: equivalence partitioning, boundary value analysis, decision tables, state transition (see Part 6).

3.4 White Box Testing (Glass Box / Structural)

What it is: Testing with **knowledge of the internal code**: statements, branches, paths and conditions. Usually done by developers.

> **Real example:** The developer reads the code of *calculateDiscount* and writes tests so every **if/else** branch runs at least once: one test for amount below 1000, one for 1000 and above, one for an invalid negative amount.

- Techniques: statement, branch/decision, condition and path coverage (see Part 6).

3.5 Grey Box Testing

What it is: A mix: the tester knows some internal details (database structure, APIs, architecture) but not the full code.

> **Real example:** The tester places an order in the UI (black box) and then checks directly in the database that the *orders* table got the correct row and stock quantity reduced by 1 (using internal knowledge).

	Black box	White box	Grey box
Knows code?	No	Yes, fully	Partly
Done by	Testers	Developers	Testers / SDETs
Focus	Behaviour / requirements	Code structure and logic	Behaviour + internal data flow
Example	Check login result	Check every if/else branch	Check UI order vs DB record

Part 4: Functional Testing Types

Functional testing checks **what the system does**: features and business rules, as per requirements. Below are all the common functional testing types.

4.1 Smoke Testing (Build Verification Testing)

What it is: A **quick, shallow check of the most critical functions** of a new build to decide if it is stable enough for deeper testing. If smoke testing fails, the build is rejected and sent back to developers. Think: 'Does the software even start and breathe?'

When it is done: Immediately after every new build/deployment.

> **Real example:** A new ShopEasy build arrives. In 15-30 minutes QA checks only: the app opens, login works, search shows results, a product can be added to the cart, the checkout page loads. If the app crashes on login, the build is rejected immediately - no further testing is wasted.

- Wide but shallow; usually automated and run on every new build.
- Done on a **new** build, often by QA or by the build pipeline (CI).

4.2 Sanity Testing

What it is: A **narrow, focused check** after a small code change or bug fix, to confirm that the change works and has not obviously broken the related area. It decides whether it is worth doing full regression testing. Think: 'Is the fix reasonable?'

When it is done: After a bug fix or small change, before full regression.

> **Real example:** Bug: 'Coupon SAVE10 does not apply.' Developer fixes it and gives build 1.0.5. The tester checks only the coupon area: SAVE10 works, an invalid coupon shows an error, and cart total still updates. It does not retest the whole site. If the coupon works, the tester then proceeds to regression testing.

- Narrow and deep (only the changed area); often unscripted and manual.
- Done on a **relatively stable** build after fixes or minor changes.

4.3 Regression Testing

What it is: Re-running existing tests to ensure that **new changes have not broken old, working features**.

When it is done: After every code change or release.

> **Real example:** A developer adds 'Pay with UPI' to ShopEasy. Regression suite re-checks old features: card payment still works, cart total is right, order history still opens, coupon still applies. Even though UPI code was added, the card payment may have accidentally broken.

- Best candidate for automation because it is repeated again and again.
- Types: **Full** regression (everything), **Partial** (only affected modules), **Unit** regression (re-run unit tests).
- Done after every change: new feature, bug fix, configuration or environment change.

4.4 Retesting (Confirmation Testing)

What it is: Running the **exact failed test case again** after the developer fixes the defect, to confirm the bug is really fixed.

> **Real example:** Test case TC-045 'Place order with 3 items' failed with bug BUG-101 (order total wrong). After the fix, the tester runs **TC-045 again** with the same data. It passes, so BUG-101 is closed.

- Retesting = the failed test again. Regression = other tests, to see nothing else broke.

4.5 Exploratory Testing

What it is: Learning, designing and executing tests **at the same time** without detailed pre-written test cases, using the tester's experience and curiosity.

> **Real example:** A tester has 1 hour with ShopEasy's new wish-list feature. They add 100 items, remove items rapidly, open the wish-list in two tabs, rotate the phone, press Back during loading - and discover a crash that no written test case covered.

- Often organised as time-boxed **sessions** with a goal (a 'test charter').

4.6 Ad-hoc Testing

What it is: Informal, unplanned testing without documentation, trying to break the app randomly. Relies on the tester's intuition. (Exploratory is more structured with charters; ad-hoc is fully random.)

> **Real example:** Tester types emojis in the address field, pastes a 5,000-character text in the search box, and double-clicks 'Pay' quickly - just to see what happens.

- **Monkey testing:** random inputs by a tool or tester with no knowledge of the app.
- **Gorilla testing:** repeatedly testing one module very heavily (e.g. login 100 times with different inputs).
- **Buddy testing:** a developer and a tester work together on a module.

4.7 End-to-End (E2E) Testing

What it is: Testing a complete **user journey from start to finish** across all systems, including external ones (payment gateway, email, SMS, database).

> **Real example:** Customer registers -> verifies email -> searches -> adds to cart -> pays via the real (sandbox) payment gateway -> receives order SMS -> warehouse system gets the order -> delivery status updates in the app.

4.8 API (Service) Testing

What it is: Testing the application's APIs directly (without the UI): requests, responses, status codes, data and error handling.

> **Real example:** Using Postman, send *GET /products?name=shoes*. Expect status **200** and a JSON list of shoes. Send *POST /cart* with a wrong product id. Expect status **404** and a clear error message. Send the request without a login token. Expect **401**.

- Faster and more stable than UI testing; finds bugs earlier.
- Tools: Postman, REST Assured, SoapUI, Karate.

4.9 UI / GUI Testing

What it is: Checking the look and behaviour of the user interface: buttons, menus, fonts, colours, alignment, links, forms and messages.

> **Real example:** On ShopEasy's cart page: the product image loads, the text is not cut off, the 'Remove' button works, the error message for an empty address is shown in red beside the field, and the layout stays correct on small screens.

4.10 Database Testing

What it is: Checking data stored in the database: correct saving, updating, deleting, constraints, transactions and data integrity.

> **Real example:** After ordering 2 pieces of an item, verify the *orders* table has a new row with the right customer id and total, and the *stock* table reduced by 2. If payment fails, verify that **no** order row remains (transaction rollback).

4.11 Interface Testing

What it is: Verifying the communication between two systems or components: data passed, formats and error handling.

> **Real example:** ShopEasy sends the order to a courier company's system. Test that the address, phone number and weight are transferred correctly, and that the app shows a friendly message if the courier system is down.

4.12 Positive and Negative Testing

What it is: Positive: use valid data and expect success. **Negative:** use invalid data and expect the system to reject it gracefully.

> **Real example:** Positive: login with the correct password -> home page. Negative: wrong password -> 'Invalid credentials' (no crash); blank username -> 'Username is required'; a SQL-like string in the username -> rejected safely.

4.13 Usability Testing

What it is: Checking how **easy, clear and pleasant** the software is for real users.

> **Real example:** Five new users are asked to buy a shirt on ShopEasy without help. The team watches: how long does it take, where do they hesitate? If 4 of 5 cannot find the 'Apply coupon' box, the design must change.

4.14 Accessibility Testing

What it is: Checking that people with disabilities can use the software (screen readers, keyboard only, colour contrast, text size).

> **Real example:** A visually impaired user uses a screen reader on ShopEasy. Every image has alt text such as 'Blue running shoes'; every button is reachable using the Tab key; text has enough contrast. Standard: WCAG guidelines.

4.15 Localization and Internationalization Testing

What it is: i18n: the app is built to support many languages and regions. **L10n**: checking a specific language/region version is correct.

> **Real example:** ShopEasy in Telugu and Hindi: text is not cut off, dates are in dd/mm/yyyy, currency shows Rs. / INR with correct separators, and special characters display properly.

4.16 Compatibility Testing (Functional side)

What it is: Checking the software works on different browsers, operating systems, devices and screen sizes. (Also listed in Part 5.)

> **Real example:** ShopEasy checkout works on Chrome, Firefox, Safari and Edge; on Android 12-15 and iOS 17-18; on small phones and large tablets.

Part 5: Non-Functional Testing Types

Non-functional testing checks **how well** the system works: speed, safety, reliability, ease of use - not what features it has.

5.1 Performance Testing (umbrella term)

What it is: Checking speed, stability and resource use of the system under a given workload. Types below all belong here. Tool: JMeter, k6, LoadRunner, Gatling.

> **Real example:** ShopEasy's search must return results in under 2 seconds and the checkout page in under 3 seconds.

Types of performance testing

Type	What it checks	ShopEasy example
Load testing	Behaviour under the **expected** number of users.	5,000 users shop at the same time on a normal day. Response time must stay under 3 sec.
Stress testing	Behaviour **beyond** the limit, until it breaks, and how it recovers.	Increase from 5,000 to 50,000 users to find the breaking point. After the load drops, does the site recover by itself?
Spike testing	A **sudden** huge jump (and drop) in users.	A flash sale starts at 12:00 and users jump from 1,000 to 40,000 within a minute.
Endurance / Soak testing	Behaviour under normal load for a **long time** (memory leaks).	Run 3,000 users for 24 hours; check memory use does not keep growing.
Volume testing	Behaviour with a **huge amount of data**.	Database with 50 million products and 20 million orders: search and reports still work.
Scalability testing	Whether adding servers/resources increases capacity as expected.	Add 2 more servers and check that capacity grows about in proportion.

5.2 Security Testing

What it is: Checking that the system protects data and resists attacks, and that only authorised people can do allowed actions. Covers confidentiality, integrity, authentication, authorization and non-repudiation.

> **Real example:** Try entering *' OR '1'='1* in the login box (SQL injection) - it must be rejected. Try *<script>alert(1)</script>* in a review box (XSS) - it must be shown as plain text. A normal user must **not** open the admin page by changing the URL. Passwords must be stored encrypted; the session must expire after inactivity.

- **Vulnerability scanning:** automated tools find known weaknesses.
- **Penetration testing:** ethical hackers actively try to break in.
- **Authentication / Authorization testing:** who you are / what you may do.
- Reference list: OWASP Top 10. Tools: OWASP ZAP, Burp Suite.

5.3 Compatibility Testing

What it is: Checking the software works across different environments: browsers, OS versions, devices, networks and hardware.

> **Real example:** A coupon box looks perfect in Chrome but is hidden behind the footer in Safari on iPhone. Compatibility testing on real devices or cloud device farms (BrowserStack) finds such issues.

- **Cross-browser testing**, **cross-platform testing**, **backward compatibility** (works with older versions), **forward compatibility** (works with future versions).

5.4 Reliability Testing

What it is: Checking that the software runs without failure for a specified time under specified conditions (consistency, stability).

> **Real example:** ShopEasy runs for 72 hours with normal activity. Measure how many failures happen. A target such as 99.9% uptime means at most about 8.7 hours of downtime in a whole year.

5.5 Recovery Testing

What it is: Checking how well the system recovers after crashes, power loss, network failure or hardware failure.

> **Real example:** Kill the server in the middle of a payment. After restart: is the customer charged twice? Is the order half-created? The system must restore a consistent state and not lose data.

5.6 Failover and Disaster Recovery Testing

What it is: Checking that a backup system takes over automatically when the main system fails, and that backups can be restored.

> **Real example:** Switch off ShopEasy's primary data centre. The standby data centre must take over within the promised time (RTO) with minimal data loss (RPO).

5.7 Installation (Deployment) Testing

What it is: Checking that the software can be installed, upgraded and uninstalled correctly.

> **Real example:** Install the ShopEasy Android app on a fresh phone, upgrade from version 2.1 to 2.2 (the user's cart and login remain), then uninstall (no leftover data).

5.8 Configuration Testing

What it is: Checking the software with different hardware/software configurations (RAM, CPU, OS settings, database versions).

> **Real example:** Run ShopEasy's admin portal on servers with 4 GB and 16 GB RAM, and with MySQL 8 and PostgreSQL 15, and compare behaviour.

5.9 Maintainability and Portability Testing

What it is: Maintainability: how easy it is to modify, fix and extend the software. **Portability:** how easily the software moves to another environment.

> **Real example:** Maintainability: change the GST percentage in one configuration file - no code change needed. Portability: move ShopEasy from one cloud provider to another and it runs with only configuration changes.

5.10 Compliance (Conformance) Testing

What it is: Checking that the software follows laws, standards and company policies.

> **Real example:** ShopEasy must follow payment-card security rules (PCI-DSS): card numbers are never stored in plain text, and personal-data privacy laws: the user can delete their account and data.

5.11 Scalability and Capacity Testing

What it is: Checking how much the system can grow and the maximum capacity it supports. (See also the table in 5.1.)

> **Real example:** Find out how many orders per minute ShopEasy can process on today's servers, so the business can plan for the festival season.

Part 6: Test Design Techniques

Techniques help testers choose a **small, smart set of test cases** instead of testing every possible input (which is impossible).

6.1 Black box techniques

Equivalence Partitioning (EP)

What it is: Divide inputs into groups (partitions) that the system treats the same. Test **one value from each group**.

> **Real example:** ShopEasy accepts age 18 to 60 for a membership form. Partitions: below 18 (invalid), 18-60 (valid), above 60 (invalid). Test values: 10, 30, 70. Three tests represent all possible ages.

Boundary Value Analysis (BVA)

What it is: Bugs often hide at the **edges**. Test values at, just below and just above each boundary.

> **Real example:** For age 18-60, test: 17 (invalid), **18** (valid), 19, 59, **60** (valid), 61 (invalid). A developer who wrote *age > 18* by mistake will be caught by the test at exactly 18.

- Tip: EP + BVA are used together in almost every project.

Decision Table Testing

What it is: Used when the result depends on **combinations of conditions**. List all combinations and the expected action.

> **Real example:** Discount rule: a member gets 10% off; a coupon gives extra 5%. See table below.

Rule	Is member?	Has coupon?	Expected discount
1	Yes	Yes	15%
2	Yes	No	10%
3	No	Yes	5%
4	No	No	0%

State Transition Testing

What it is: Used when the system has **states** and moves between them on events. Test valid and invalid transitions.

> **Real example:** ATM PIN: Attempt 1 wrong -> Attempt 2 wrong -> Attempt 3 wrong -> **Card blocked**. A correct PIN at any time moves to 'Authenticated'. Tests: 2 wrong then correct (should work); 3 wrong (card must be blocked); after block, even the correct PIN must fail.

- ShopEasy order states: Placed -> Paid -> Packed -> Shipped -> Delivered. An order cannot go from 'Placed' directly to 'Delivered'.

Use Case Testing

What it is: Designing tests from real-life user scenarios (use cases) with main flow and alternate flows.

> **Real example:** Use case 'Book train ticket': Main flow: search trains -> choose train -> enter passengers -> pay -> ticket generated. Alternate flows: seats unavailable (waiting list), payment fails (retry), session timeout during payment.

Error Guessing and Pairwise (Combinatorial) Testing

What it is: Error guessing: testers use past experience to guess where bugs may be. **Pairwise:** test every pair of parameter values instead of every full combination - massively reducing test cases.

> **Real example:** Error guessing: blank fields, spaces only, very long text, special characters, zero, negative numbers, past dates. Pairwise: 4 browsers x 3 OS x 3 languages = 36 combinations, but pairwise covers every pair of values with about 12 tests.

6.2 White box (coverage) techniques

Take this simple code:

> **Code:** `if (amount >= 1000) { discount = 10%; } else { discount = 0%; }`

Technique	Meaning	Example on the code above
Statement coverage	Every line of code runs at least once.	Need 2 tests: amount=1500 (runs the if line) and amount=500 (runs the else line).
Branch / Decision coverage	Every True and False outcome of every decision runs.	amount=1500 (True) and amount=500 (False) give 100% branch coverage.
Condition coverage	Each individual condition in a compound decision is True and False.	For *(age > 18 AND hasID)*: test age true/false and hasID true/false separately.
Path coverage	Every possible path through the code is executed.	With 3 independent if-statements there are 8 paths; test all 8.
Loop testing	Test loops with 0, 1, many and maximum iterations.	Cart with 0 items, 1 item, 10 items and the maximum 99 items.

Part 7: Other Important Testing Types

7.1 Alpha Testing and Beta Testing

What it is: Alpha: done by internal employees (not the dev team) in a controlled environment. **Beta:** done by a limited group of real external users in the real environment before the public release.

> **Real example:** Alpha: ShopEasy staff from other departments use the new app for two weeks and report bugs. Beta: 2,000 real customers get the new app from the Play Store 'beta programme' and send feedback.

7.2 A/B Testing

What it is: Showing two versions (A and B) to different groups of real users and measuring which performs better.

> **Real example:** Group A sees a green 'Buy now' button, Group B sees an orange one. After a week, B gets 8% more purchases, so B becomes the default.

7.3 Static Testing (Reviews)

What it is: Finding defects **without running** the code: reviewing documents, requirements, designs and code, or using static analysis tools.

> **Real example:** Requirement says 'The system must be fast.' In a review the tester raises a defect: *fast* is not measurable. It is rewritten as 'Search results appear in under 2 seconds for 95% of requests.'

- **Informal review:** a colleague reads and comments.
- **Walkthrough:** the author explains the document step by step to a group.
- **Technical review:** experts check it against standards.
- **Inspection:** the most formal, with defined roles and checklists.
- **Dynamic testing** is the opposite: the software is executed.

7.4 Mutation Testing

What it is: Deliberately introducing small changes (mutants) into the code to check if the existing tests detect them. It measures the quality of your tests.

> **Real example:** The tool changes *amount >= 1000* to *amount > 1000*. If all your unit tests still pass, your tests are weak because they did not detect the change; add a test for exactly 1000.

7.5 Fuzz Testing (Fuzzing)

What it is: Sending large amounts of random, malformed or unexpected data to the software to find crashes and security holes.

> **Real example:** A tool sends thousands of random strings, huge files and broken JSON to ShopEasy's upload-profile-photo API to see whether the server crashes or leaks data.

7.6 Chaos Testing (Chaos Engineering)

What it is: Deliberately breaking parts of a production-like system (killing servers, cutting the network) to check it stays resilient.

> **Real example:** Randomly shut down one of ShopEasy's ten servers during peak time. Customers should not even notice because the load balancer shifts traffic to the others.

7.7 Contract Testing

What it is: Checking that two services (consumer and provider) agree on the format of requests and responses (their 'contract'), common in microservices.

> **Real example:** The Cart service expects the Product service to return *price* as a number. If the Product team changes it to text '499 INR', the contract test fails before release and prevents a production break.

7.8 Mobile App Testing

What it is: Testing apps on phones and tablets, considering devices, OS versions, touch gestures, battery, network changes and interruptions.

> **Real example:** While paying in the ShopEasy app: a phone call arrives, the network switches from Wi-Fi to 4G, battery goes low, the user rotates the screen. The payment must not be lost or duplicated. Tool: Appium; real devices or emulators.

7.9 Data-Driven and Keyword-Driven Testing

What it is: Data-driven: the same test script runs with many sets of data from a file/table.
Keyword-driven: tests are written using keywords like Login, Search, AddToCart.

> **Real example:** One login script reads 50 rows (username, password, expected result) from an Excel/CSV file and runs 50 tests automatically.

7.10 Installation, Migration and Upgrade Testing

What it is: Checking that data and functions are correct after moving to a new system, database or version.

> **Real example:** ShopEasy moves from an old database to a new cloud database. Check that all 2 million customer records, order histories and balances match exactly before and after migration.

7.11 Globalization, Cloud, IoT and Game Testing (overview)

What it is: Specialised testing for specific technologies.

> **Real example: Cloud:** auto-scaling works when load grows. **IoT:** a smart door lock opens only for the right phone, even with a weak network. **Game:** frame rate, graphics, controls and in-app purchases on different devices.

7.12 Software Quality Terms Often Confused

What it is: A few more terms students meet in interviews.

> **Real example: Pilot testing:** a small real-life trial before full roll-out. **Beta:** external user trial. **Acceptance:** customer sign-off. **Smoke:** is the build stable? **Sanity:** is the fix OK? **Regression:** did anything old break?

Part 8: Smoke vs Sanity vs Regression vs Retesting

This is the most common interview topic. Use the ShopEasy story: *Build 1.0 arrives -> smoke test -> bug found in coupon -> fix -> retest -> sanity -> regression.*

Point	Smoke	Sanity	Regression	Retesting
Purpose	Is the new build stable enough to test?	Is the fix/change working properly?	Did new changes break old features?	Is the specific defect really fixed?
Scope	Wide and shallow (main features)	Narrow and deep (changed area)	Wide and deep (existing features)	Only failed test cases
Build	New, unstable build	Relatively stable build after fix	Stable build	Build with the defect fix
Scripted?	Usually scripted / automated	Usually unscripted	Scripted, mostly automated	Uses the same failed test case
Done by	Developers or testers / CI pipeline	Testers	Testers (automation team)	Testers
If it fails	Build rejected	Fix rejected, regression skipped	Defects raised for broken features	Defect re-opened
ShopEasy example	App opens, login, search, add to cart, checkout page loads	Coupon SAVE10 now works and cart total updates	Card payment, order history, address book still work after UPI added	Run TC-045 again with the same data

> **Memory trick:** Smoke = **'Is it on fire?'** (basic health of a new build). Sanity = **'Is it sane?'** (does the fix make sense). Regression = **'Did we go backwards?'**. Retesting = **'Run the failed test again.'**

Other pairs students confuse

Pair	Difference
Verification vs Validation	Verification = reviews/inspections without running code (are we building it right?). Validation = running the software (are we building the right thing?).
Load vs Stress	Load = expected users. Stress = beyond the limit to find the breaking point.
Alpha vs Beta	Alpha = internal users at the company. Beta = real external users at their own place.
Black vs White box	Black = no code knowledge, tests behaviour. White = code knowledge, tests structure.
Functional vs Non-functional	Functional = what the system does. Non-functional = how well it does it (speed, security, usability).
Exploratory vs Ad-hoc	Exploratory = structured learning + testing with a goal. Ad-hoc = random and informal.
Static vs Dynamic	Static = no execution (reviews). Dynamic = software is executed.
Error vs Defect vs Failure	Error = human mistake. Defect = fault in code. Failure = wrong behaviour seen by the user.

Part 9: Defect Life Cycle, Severity vs Priority, Test Case Format

Bug (defect) life cycle

New -> **Assigned** (to a developer) -> **Open** (developer starts work) -> **Fixed** -> **Retest** -> **Verified / Closed** (if the fix works) or **Reopened** (if it still fails). Other possible states: **Rejected** (not a bug), **Duplicate**, **Deferred** (fix in a later release).

Severity vs Priority

Severity = how serious the bug is for the system (technical impact, set by the tester). **Priority** = how soon it must be fixed (business urgency, set by the product owner/manager).

Case	Severity	Priority	Example
High severity, high priority	High	High	ShopEasy checkout crashes - no one can buy.
High severity, low priority	High	Low	App crashes only on a very old phone model that 0.1% of users use.
Low severity, high priority	Low	High	Company name misspelled on the home page before the launch event.
Low severity, low priority	Low	Low	Slight colour difference of a footer link.

Sample bug report

Field	Example
Bug ID / Title	BUG-101 - Order total is wrong when coupon SAVE10 is applied to 3 items
Environment	ShopEasy web, Chrome 130, Windows 11, build 1.0.4
Steps to reproduce	1) Login. 2) Add 3 items totalling Rs. 1500. 3) Apply coupon SAVE10. 4) Open the cart summary.
Expected result	Total = Rs. 1350 (10% discount).
Actual result	Total = Rs. 1500 - discount not applied (screenshot attached).
Severity / Priority	High / High
Status	New

Sample test case

Field	Example
Test case ID	TC-LOGIN-001
Title	Login with valid credentials
Pre-condition	User 'ravi@example.com' is registered and active.
Test steps	1) Open ShopEasy login page. 2) Enter email 'ravi@example.com'. 3) Enter the correct password. 4) Click Login.
Test data	Email: ravi@example.com; Password: (valid)
Expected result	User lands on the home page and sees 'Welcome, Ravi'.
Actual result / Status	(filled during execution) Pass / Fail

Part 10: Quick Revision Table and Interview Questions

Testing type	One-line meaning	Quick example
Unit	Test one function.	calculateDiscount(1000) = 100
Integration	Test modules together.	Cart total reaches the Payment module correctly
System	Test the whole app.	Full order journey
Acceptance (UAT)	Customer approves.	Business team tries return/refund flow
Smoke	Is the new build stable?	App opens, login, search, checkout page
Sanity	Is the fix OK?	Coupon works after bug fix
Regression	Did old features break?	Card payment after adding UPI
Retesting	Run the failed test again.	Re-run TC-045
Exploratory	Learn and test at once.	Wish-list free exploration for 1 hour
Ad-hoc / Monkey	Random, unplanned testing.	Pasting 5,000 characters in search
API	Test services directly.	GET /products returns 200
Load	Expected number of users.	5,000 shoppers
Stress	Beyond the limit.	50,000 users
Spike	Sudden jump of users.	Flash sale at 12:00
Soak / Endurance	Long duration.	24 hours at normal load
Security	Protect data and access.	SQL injection rejected
Usability	Ease of use.	New users find the coupon box
Compatibility	Different browsers/devices.	Chrome, Safari, Android, iOS
Recovery	Recovers after failure.	Server crash during payment
Localization	Region/language correct.	Telugu and Hindi versions
Accessibility	Usable by disabled users.	Screen reader support
Alpha / Beta	Internal / external trial.	Staff trial then Play Store beta
A/B	Compare two versions.	Green vs orange 'Buy now' button
Mutation	Check test quality.	Change >= to > and see tests fail
Fuzz	Random bad data.	Random files sent to upload API
Chaos	Break parts on purpose.	Kill one server at peak time

Common interview questions

- What is the difference between smoke testing and sanity testing?
- What is the difference between regression testing and retesting?
- What is the difference between load, stress and spike testing?
- Explain black box, white box and grey box testing with examples.
- What is boundary value analysis? Give an example for an age field 18 to 60.

- What is the difference between severity and priority? Give an example of high severity and low priority.
- What is the difference between verification and validation?
- What is the bug life cycle?
- Which test cases would you automate and which would you keep manual?
- How do you test a login page? (Hint: valid and invalid inputs, blank fields, boundary lengths, SQL injection, lock after 3 wrong attempts, password masked, session timeout, browser Back button, different browsers, performance with many users.)

Practice exercise for students

Choose a train-ticket booking website. For each of these, write **two test cases**: smoke, sanity, regression, boundary value (number of passengers 1 to 6), negative (past travel date), load (peak booking time), security (password), usability and compatibility. Then classify one imaginary bug by severity and priority.

> **Final tip:** In real projects the types are combined: a new build gets **smoke**; fixes get **retesting + sanity**; every release gets **regression**; before launch you run **performance, security, compatibility and UAT**. Learn *when* to use each type, not just the definition.